第 7 章 和 AI 对话：把需求说清楚

> 本章目标：掌握"目标 / 背景 / 限制 / 验收"四要素写法，学会用三段式报错模板描述现状，知道怎样迭代才不会被 AI 越改越乱。

> 要点：AI 不是"读心术"，而是"照字面执行"。你没写出来的东西，它只能替你猜；它猜的往往是一个平庸但合理的默认值。

7.1 为什么"帮我做个网站"一定会翻车

你在聊天框里打一句"帮我做个网站"，模型会立刻开始写代码。写出来的东西看起来挺像，但打开一用——配色不对、没有登录、数据一刷新就丢、手机上排版全乱。

问题不在模型笨。问题在于：**"网站"背后是一整个巨大的需求空间**。给谁用？做几页？数据放哪？要不要登录？部署在哪？这些模型都不知道，只能按训练数据里"最常见的那种网站"去填，而那个"最常见"和你的真实需求基本不可能重合。

换个角度：问同事"帮我做个网站"，对方会反问你五个问题；问 AI，它直接开工。**它不问，你就得自己先说。**

与人沟通 vs 与 AI 沟通

维度	和人沟通	和 AI 沟通
缺失信息	凭常识、经验、行业习惯补上	只能按你给的上下文补，补不上就编
情绪线索	能看表情、听语气	完全不读情绪
主动追问	会问"这个给谁用？"	通常不主动问，除非你明确要求
礼貌话术	有社交作用	不增加任何信息量
结构化信息	有效，但人会烦	有效，且越结构化越好
重复与耐心	反复讲同一件事人会觉得烦	不会厌倦，同样认真地答

维度	和人沟通	和 AI 沟通
被带偏	会被你的态度影响	会被之前放进上下文的内容影响（错误前提、矛盾要求）

结论很清楚：**对 AI，礼貌不加工资，结构才是工资。** 把话写清楚、分成条目、给出验收标准，比用什么修辞都管用。

7.2 四要素：目标 / 背景 / 限制 / 验收

这是本章最值得背下来的东西。任何需求，缺了这四块中的任何一块，AI 都只能猜。

- **目标**：你要达成什么结果。不是"写个页面"，而是"家长能在手机上看到孩子今天的作业提交状态"。
- **背景**：现在是什么情况——技术栈、已有文件、谁在用、试过什么。
- **限制**：不能做什么。不许引入新依赖、不许动某个文件、必须是纯前端、必须兼容什么浏览器。
- **验收**：怎么判断做完了。可操作、可观察、最好能量化。

改写示例：家庭通讯软件的一个功能

改之前（失败版）

> 帮我写一个家庭群聊软件的消息已读功能。

这句话的问题：什么技术栈？已读是显示文字还是一排头像？群聊里给所有人看还是只给发送者看？没有服务器怎么办？

改之后（四要素版）

> **目标**：在现有的家庭群聊页面里，让发送者能看到自己发出的消息被哪些家庭成员读过。
>
> **背景**：项目在 `web/chat/`，前端是纯 HTML + 原生 JavaScript，后端是 `server.js`（Node，暂用内存存储，没有数据库）。目前消息列表的渲染在 `chat.js` 的 `renderMessages()` 里，用户身份存在 `currentUser` 变量中。
>
> **限制**：不引入新的前端框架和 CDN 依赖；不改动已有的登录逻辑；消息记录最多保留 200 条，单条消息的已读列表最多记 50 人。
>
> **验收**：

1. 用户 A 发出消息后，气泡右下角显示"未读"；
2. 用户 B 打开该会话并滚动到这条消息，A 的界面在 3 秒内变成"已读 1 人"；
3. 把浏览器窗口缩到手机宽度，已读提示不换行、不遮挡正文；
4. 断网时显示"未同步"，不要报错崩溃。

先不要写代码，先告诉我你打算怎么实现，以及有哪几处你觉得我的描述还不够明确。

注意最后那一句——它顺手解决了 7.4 节要讲的问题。

7.3 描述现状：三段式报错模板

AI 看不到你的屏幕，也看不到控制台。它只能读你发给它的文字。所以报错要像写病历一样写：**现象 / 复现 / 尝试**。

```
【现象】运行 python -m http.server 8000 后打开页面，控制台报
  Uncaught TypeError: Cannot read properties of null (reading 'value')
  出现在 chat.js 第 42 行。页面上发送按钮点下去没有反应，输入框里的字不消失。

【复现】1. 启动服务；2. 打开 /chat.html；3. 在输入框输入任意文字；4. 点击"发送"。

【已尝试】
  - 把第 42 行改成 (el || {}).value，按钮有反应了，但消息内容变成 undefined；
  - 查过 DOM 里确实有 id="msgInput" 的元素，所以不是选择器写错；
  - 换成 Chrome 和 Edge 都一样，排除浏览器问题。

【期望 / 实际】期望点击发送后消息出现在列表里。实际是按钮无响应，控制台报上面的错。
```

这个模板为什么有效：**"现象"给出事实，"已尝试"排除掉 AI 会重复走的路**。少了后一段，AI 大概率会建议你"先检查选择器"——而你早就查过了，白问一轮。

配套的习惯：贴**报错的原文**而不是转述（原文里有行号和调用栈）；贴**相关文件的内容或路径**；说明**期望和实际**的差别，而不是只说"不工作"。

7.4 迭代方式：小步、可回退、要证据

初学者最常见的失败模式不是"问得不好"，而是"一次改太多，改坏了不知道是哪一处坏"。

先小后大。 需求先拆成能独立验证的小块：先让它把页面渲染出来、能点、能存；再加已读；再加同步。每块都能单独跑到"看得见"的状态。

一次一改。 一轮对话只让它改一件事。改了十处、结果不对，你没法定位。

让它先复述。 动手之前加一句："请先用三句话复述我的需求，并列出你不确定的地方。"这一步能拦掉大量误解。

要求 diff，不要整文件。 说"请以 diff 形式给出改动，只展示改动的行"。整文件重写会顺手改掉你没要求改的地方。

要求测试证据。 "请贴出运行命令和完整输出。"这是区分"说得出"和"跑得通"的唯一办法（第八章展开）。

大任务先出方案。 超过三五个文件的任务，先让它给改动清单，你确认了再动手。

7.5 提示词工程：有用的部分

关于"提示词工程"，网上有很多玄乎的说法。以教学经验看，真正稳定有效的只有几条：

结构化 > 修辞。 分条目的需求、明确的字段名、编号的验收标准，效果稳定；华丽的描述性语言基本无效。

给例子（few-shot）。 要它按某种格式输出，就给一两个"输入 → 期望输出"的样例，比用文字描述格式准确得多。 类比：教人做菜，说十句"要恰到好处"不如直接给一张成品照片。

写清约束和格式。 "只输出 JSON，不要解释""不超过 200 字"。不写，它就按自己的习惯输出。

角色设定没那么重要。 很多教程强调"你是一个资深工程师"之类的开场。以我的体验，它的作用远小于上下文和验收标准——告诉它"这是一个跑在 RK3588 上的脚本，只有 2GB 可用内存"，比告诉它"你是嵌入式专家"有用得多。**这是经验判断，不是普遍规律。**

7.6 让 AI 反问、让 AI 挑战你

模型默认是"顺从"的：你说什么它都倾向配合。所以有两个反向技巧特别有用：

一、让它列缺口。

> 先不要写代码。列出为了实现这个需求，你还缺哪些信息；哪些信息缺失会显著影响实现方式。

它的回答通常会暴露你自己没想到的点：并发、权限、数据量、失败重试。

二、让它反驳你（grilling）。

> 这是我的方案：[描述]。请扮演一个挑刺的评审，指出这个方案最可能失败的三个地方，以及你会怎么测试它。不要客气。

这个技巧特别值的时候：方案涉及真实设备（错了有物理后果）、要给别人演示怕被问住、你已经"觉得"没问题了（这时最容易漏掉盲区）。

但它也有边界：AI 挑的刺同样可能是幻觉，它编出来的"风险"未必真实存在。把它当成**提问清单**，而不是**判决书**。

7.7 常见误区

误区	为什么不行	改法
超长 prompt，把十件事写在一起	模型会顾此失彼，你不知道哪条没执行	拆成多轮，一轮一件事
堆砌术语（"请用 OOP 和 SOLID 原则……"）	术语不提供具体信息，反而挤占了真正的要求	换成具体约束：文件结构、命名、不许用什么
只描述愿望，不描述验收	做完了没法判断对错	补上可观察的完成标准
一次要求改十处	出错了无法定位	一次一改，改完立刻验证
把 AI 当搜索引擎	它不知道的事会编，且编得很像真的	事实类问题查一手来源（第八章）
情绪化催促（"你怎么又错了！"）	不增加信息量，还可能让它迎合你而放弃正确判断	贴出实际输出和期望输出的差别
反复"再来一次"却不改描述	同样的输入，大概率得到同样的结果	改需求描述或补上下文，而不是抽卡

课后题

1. 下面是原始需求，请用"目标 / 背景 / 限制 / 验收"四要素改写成一份完整需求： > 帮我做一个点名的小程序。 （要求：验收标准至少 3 条，且每条都是可观察的。）
2. 找一段你最近发给 AI 的 prompt，用 7.7 节的表格给它挑刺，写出至少两个问题。
3. 用三段式模板写一份报错描述（现象 / 复现 / 已尝试），要求"已尝试"里至少有两条真实的排查记录。
4. 拿你小组的项目方案，用 7.6 节的"挑战"提示词让 AI 挑三个最可能失败的地方，然后逐条判断：哪些是真风险，哪些是它编的？说明判断依据。
5. 开放题：有人说"对 AI 说话越像对人说话越好"。结合 7.1 节的对照表，谈谈你同意和不同意的部分。